

Data Science Bootcamp Written Assessment

SECTION A

Q1: What is Jupyter Notebook and how is it used in data science?

Jupyter Notebook is an open-source, browser-based interactive computing environment that lets you combine live code, its output (tables, charts), equations and narrative text (Markdown) in a single document (.ipynb). In data science it is used to load and clean data, run exploratory analysis cell by cell, visualise results inline, prototype and evaluate machine learning models, and share reproducible analyses or reports with others.

Q2 Name the steps in a data science life cycle

1. Problem definition / business understanding – define the question and success criteria.
2. Data collection – gather data from files, databases, APIs or web scraping.
3. Data cleaning / preparation – handle missing values, duplicates, wrong data types and outliers.
4. Exploratory data analysis (EDA) – summarise and visualise the data to find patterns and relationships.
5. Feature engineering – select, transform, encode and scale features.
6. Modelling – choose and train machine learning models.
7. Model evaluation – measure performance with suitable metrics and tune the model.
8. Deployment – put the model into production or communicate the results.
9. Monitoring and maintenance – track performance over time and retrain as needed.

SECTION B

Q1: How do you reverse a string in Python?

Use slicing with a step of -1:

```
s = "hello"
```

```
reversed_s = s[::-1] # 'olleh'
```

Alternatively: `''.join(reversed(s))`

Q2: How do you access the last element of a list in Python?

Use negative indexing:

```
my_list = [10, 20, 30]
```

```
last = my_list[-1] # 30
```

`(my_list[len(my_list) - 1]` also works; `my_list.pop()` returns and removes it.)

SECTION C

Q1: How do you import a module in Python?

Use the import statement:

```
import math          # import the whole module
import numpy as np    # import with an alias
from math import sqrt # import a specific function
from os import path, getcwd # import several names
```

Q2: What is a NumPy array?

A NumPy array (ndarray) is NumPy's core data structure: a fixed-size, n-dimensional grid of elements that all share the same data type, stored in contiguous memory.

Compared with Python lists it is much faster and more memory-efficient, and it supports vectorised (element-wise) operations, broadcasting, slicing and a large library of mathematical, statistical and linear-algebra functions.

Example: `import numpy as np; a = np.array([1, 2, 3]); a * 2 -> array([2, 4, 6])`

SECTION D

Q1: What is the purpose of exploratory data analysis?

Exploratory data analysis (EDA) is used to understand a dataset before modelling. Its purpose is to summarise the main characteristics of the data (distributions, central tendency, spread), detect missing values, errors and outliers, uncover patterns, trends and relationships/correlations between variables, check assumptions, and guide feature selection, data cleaning and the choice of model. It is typically done with summary statistics (`describe()`, `info()`) and visualisations (histograms, box plots, scatter plots, heatmaps).

Q2: What is matplotlib, and how is it used in data science?

Matplotlib is Python's foundational 2-D plotting library (mainly used through its `pyplot` module).

In data science it is used to visualise data during EDA and to communicate results – e.g. line charts, bar charts, histograms, scatter plots, box plots and heatmaps – and to plot model results such as predictions vs. actuals or residuals. Libraries such as `seaborn` and `pandas plotting` are built on top of it.

Example: `import matplotlib.pyplot as plt; plt.scatter(x, y); plt.xlabel('x'); plt.show()`

SECTION E

Q1: What is a correlation?

A correlation is a statistical measure of the strength and direction of the linear relationship between two variables, usually expressed by the Pearson correlation coefficient r , ranging from -1 to +1. +1 is a perfect positive relationship (both increase together), -1 a perfect negative relationship (one rises as the other falls), and 0 no linear relationship. Correlation does not imply causation. In pandas it is computed with `df.corr()`.

Q2: What is linear regression?

Linear regression is a supervised machine learning algorithm used to predict a continuous target variable by modelling it as a linear combination of one or more input features:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

where b_0 is the intercept and $b_1..b_n$ are coefficients. The model finds the line (or hyperplane) of best fit by minimising the sum of squared errors between predicted and actual values (Ordinary Least Squares). Simple linear regression uses one feature; multiple linear regression uses several. It is evaluated with metrics such as R^2 , MSE and RMSE – e.g. predicting car price from horsepower, engine size and other features. In Python: `sklearn.linear_model.LinearRegression`.

SECTION F

Q1: What is classification, and how is it performed in Python?

Classification is a supervised learning task where a model learns from labelled data to predict which discrete category (class) a new observation belongs to. In Python it is typically performed with scikit-learn: load the data, split into features X and labels y , do a train/test split (`train_test_split`), scale features if needed (`StandardScaler`), choose a classifier (e.g. `LogisticRegression`, `DecisionTreeClassifier`, `KNeighborsClassifier`, `RandomForestClassifier`, `SVC`), train it with `.fit(X_train, y_train)`, predict with `.predict(X_test)`, and evaluate with `accuracy_score`, `confusion_matrix` and `classification_report`.

Q2: What is a classification problem?

A classification problem is a problem where the output (target) variable is categorical rather than continuous, so the goal is to assign each input to one of a set of predefined classes. It can be binary (two classes, e.g. spam/not spam, malignant/benign tumour) or multi-class (more than two, e.g. classifying types of flowers or digits 0-9).

SECTION G

Q1: What is the purpose of hyperparameter tuning?

Hyperparameters are model settings chosen before training rather than learned from the data (e.g. `max_depth` of a decision tree, `C` and `penalty` in logistic regression, `n_neighbors` in KNN).

Hyperparameter tuning searches for the combination of these settings that gives the best model performance on unseen data, improving accuracy and controlling overfitting/underfitting. It is usually done with cross-validation using methods such as `GridSearchCV` (tries every combination) or `RandomizedSearchCV` (samples combinations).

Q2: What is a confusion matrix?

A confusion matrix is a table used to evaluate a classification model by comparing predicted labels against actual labels. For binary classification it has four cells:

True Positives (TP) – positive cases correctly predicted positive

True Negatives (TN) – negative cases correctly predicted negative

False Positives (FP) – negative cases wrongly predicted positive (Type I error)

False Negatives (FN) – positive cases wrongly predicted negative (Type II error)

From it you can calculate accuracy, precision, recall and F1-score. In Python:
`sklearn.metrics.confusion_matrix(y_test, y_pred)`.